

Nesneye Yönelik Programlama

Hafta 9

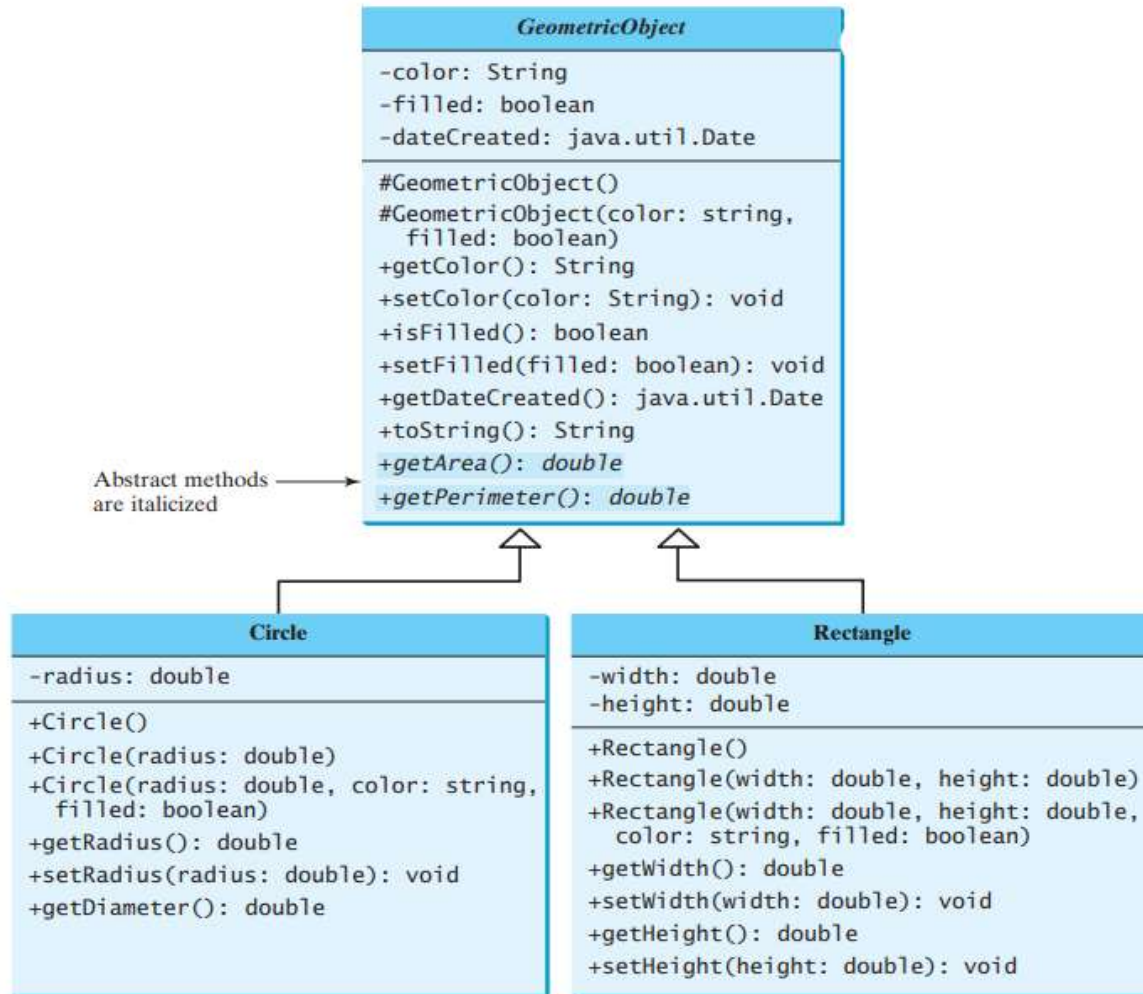
Abstract Sınıflar

Birine "Bana bir *Hayvan* çiz" derseniz, çizemez. Ne çizeceğini sorar; Kedi mi, Köpek mi, Kuş mu?

İşte burada **Hayvan** soyut bir kavramdır (Abstract Class). **Kedi** ve **Köpek** ise somut nesnelerdir (Concrete Class).

Soyut sınıfların 2 temel amacı vardır:

1. **Ortak Özellikleri Tek Yerde Toplamak:** Bütün alt sınıfların sahip olacağı ortak özellikleri ve metotları bir kere yazıp kod tekrarını (Kediye ayrı, köpeğe ayrı yaş değişkeni tanımlamayı) önlemek.
2. **Kendisinden Nesne Üretilmesini Engellemek:** Mantıken havada kalan, sadece şablon olması gereken bir kavramdan **new** anahtar kelimesi ile nesne yaratılmasını engellemek (**new Hayvan()** yapılamaz).



Abstract Class Örneği

```
1 public abstract class GeometricObject {
2     private String color = "white";
3     private boolean filled;
4     private java.util.Date dateCreated;
5
6     /** Construct a default geometric object */
7     protected GeometricObject() {
8         dateCreated = new java.util.Date();
9     }
10
11    /** Construct a geometric object with color and filled value */
12    protected GeometricObject(String color, boolean filled) {
13        dateCreated = new java.util.Date();
14        this.color = color;
15        this.filled = filled;
16    }
17
18    /** Return color */
19    public String getColor() {
20        return color;
21    }
22 }
```

Abstract Class Örneği

```
21     }
22
23     /** Set a new color */
24     public void setColor(String color) {
25         this.color = color;
26     }
27
28     /** Return filled. Since filled is boolean,
29     * the get method is named isFilled */
30     public boolean isFilled() {
31         return filled;
32     }
33
34     /** Set a new filled */
35     public void setFilled(boolean filled) {
36         this.filled = filled;
37     }
38
39     /** Get dateCreated */
40     public java.util.Date getDateCreated() {
41         return dateCreated;
42     }
```

Abstract Class Örneği

```
43
44  @Override
45  public String toString() {
46      return "created on " + dateCreated + "\ncolor: " + color +
47          " and filled: " + filled;
48  }
49
50  /** Abstract method getArea */
51  public abstract double getArea();
52
53  /** Abstract method getPerimeter */
54  public abstract double getPerimeter();
55  }
```

Abstract Class Örneği

```
1 public class Circle extends GeometricObject {  
2     // Same as lines 3-48 in Listing 11.2, so omitted  
3 }
```

```
1 public class Rectangle extends GeometricObject {  
2     // Same as lines 3-51 in Listing 11.3, so omitted  
3 }
```

```
1 public class TestGeometricObject {
2     /** Main method */
3     public static void main(String[] args) {
4         // Create two geometric objects
5         GeometricObject geoObject1 = new Circle(5);
6         GeometricObject geoObject2 = new Rectangle(5, 3);
7
8         System.out.println("The two objects have the same area? " +
9             equalArea(geoObject1, geoObject2));
10
11         // Display circle
12         displayGeometricObject(geoObject1);
13
14         // Display rectangle
15         displayGeometricObject(geoObject2);
16     }
17
18     /** A method for comparing the areas of two geometric objects */
19     public static boolean equalArea(GeometricObject object1,
20                                     GeometricObject object2) {
21         return object1.getArea() == object2.getArea();
22     }
23
24     /** A method for displaying a geometric object */
25     public static void displayGeometricObject(GeometricObject object) {
26         System.out.println();
27         System.out.println("The area is " + object.getArea());
28         System.out.println("The perimeter is " + object.getPerimeter());
29     }
30 }
```


Ekran Çıktısı

```
The two objects have the same area? false
```

```
The area is 78.53981633974483
```

```
The perimeter is 31.41592653589793
```

```
The area is 13.0
```

```
The perimeter is 16.0
```

Comparator Interface

Java'da sayılardan veya metinlerden (String) oluşan bir listeyi sıralamak kolaydır; çünkü Java bunların doğal sıralamasını (küçükten büyüğe veya alfabetik) bilir.

Ancak kendi oluşturduğumuz sınıflardan (örneğin **Ogrenci** sınıfı) nesneler içeren bir listeyi sıralamak istediğimizde Java'ya **hangi kritere göre** sıralama yapacağını söylememiz gerekir.

Comparator Interface

Öğrencileri numarasına göre mi, adına göre mi, yoksa not ortalamasına göre mi sıralayacağız? İşte **Comparator** arayüzü tam bu noktada devreye girer.

Orijinal **Ogrenci** sınıfının kodunu hiç değiştirmeden, **farklı sıralama kuralları (kriterleri)** oluşturmamıza olanak tanır.

Comparator Interface Nasıl Çalışır?

`Comparator` arayüzünü uygulayan (implement eden) bir sınıfın ezmesi (override etmesi) gereken tek bir önemli metod vardır: `compare(T o1, T o2)`.

Bu metod iki nesneyi karşılaştırır ve geriye `int` tipinde bir değer döndürür:

- **Negatif değer (< 0):** Birinci nesne (o1), ikinci nesneden (o2) küçüktür (o1 başa geçer).
- **Sıfır (0):** İki nesne birbirine eşittir.
- **Pozitif değer (> 0):** Birinci nesne (o1), ikinci nesneden (o2) büyüktür (o2 başa geçer).

```
1 // 1. OGRENCI SINIFI (SADECE VERI TUTAR)
2 class Ogrenci {
3     int numara;
4     String isim;
5
6     public Ogrenci(int numara, String isim) {
7         this.numara = numara;
8         this.isim = isim;
9     }
10
11     public String toString() {
12         return numara + " - " + isim;
13     }
14 }
```

```
16 // 2. NUMARAYA GORE SIRALAYAN COMPARATOR
17 class NumaraSiralama implements Comparator<Ogrenci> {
18     public int compare(Ogrenci o1, Ogrenci o2) {
19         return o1.numara - o2.numara; // kucukten buyuge
20     }
21 }
22
23 // 3. ISME GORE SIRALAYAN COMPARATOR
24 class IsimSiralama implements Comparator<Ogrenci> {
25     public int compare(Ogrenci o1, Ogrenci o2) {
26         return o1.isim.compareTo(o2.isim); // alfabetik
27     }
28 }
```

```
30 public class Test {
31     public static void main(String[] args) {
32         ArrayList<Ogrenci> list = new ArrayList<>();
33         list.add(new Ogrenci(3, "Ayse"));
34         list.add(new Ogrenci(1, "Zeynep"));
35         list.add(new Ogrenci(2, "Mehmet"));
36
37         // Numaraya gore sirala
38         Collections.sort(list, new NumaraSiralama());
39         System.out.println(list); // [1 - Zeynep, 2 - Mehmet, 3 - Ayse]
40
41         // Isme gore sirala (Ayni listeyi farkli siralayabiliriz!)
42         Collections.sort(list, new IsimSiralama());
43         System.out.println(list); // [3 - Ayse, 2 - Mehmet, 1 - Zeynep]
44     }
45 }
```